IOT
ESP8266

Paulo R T Cândido

ESP8266

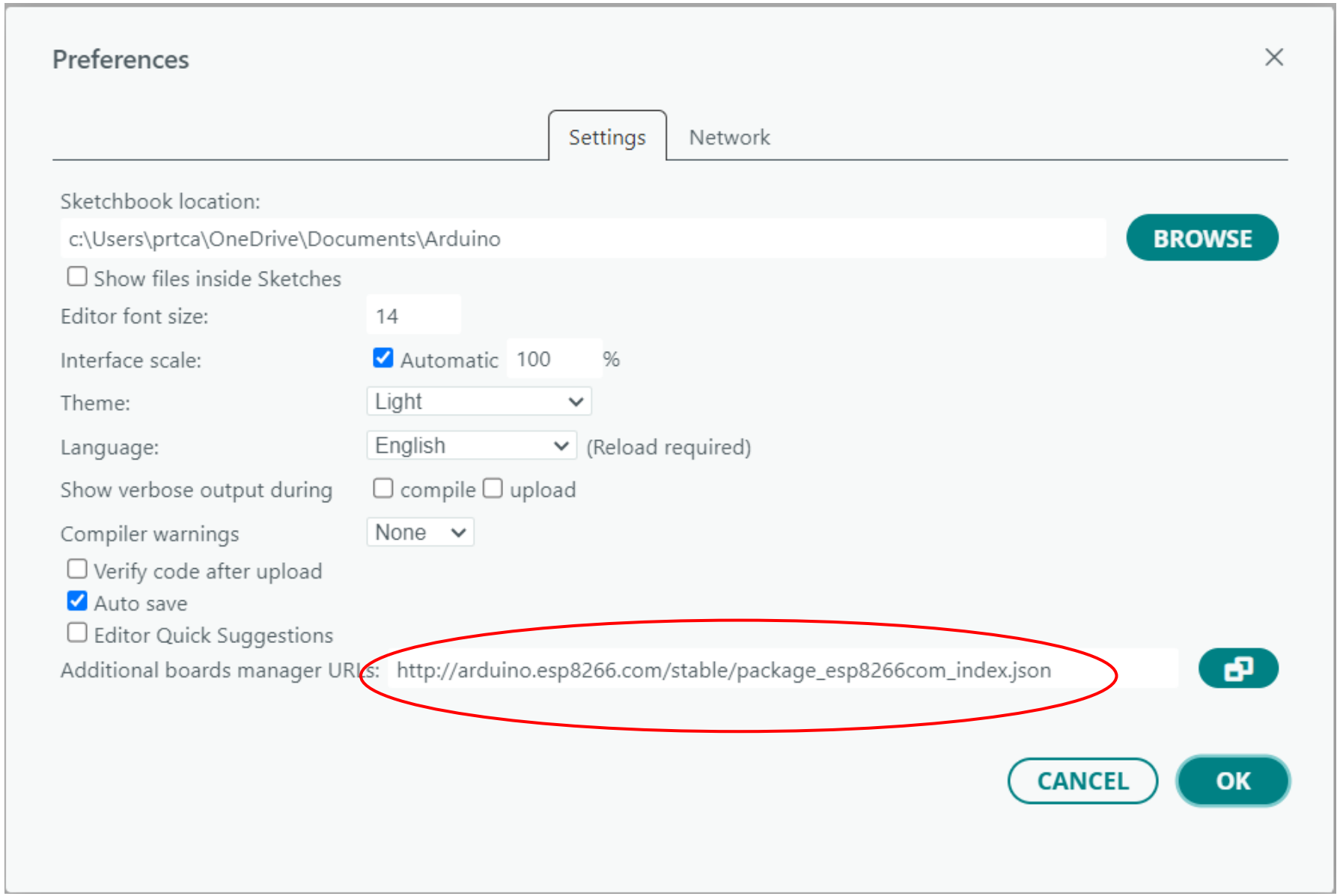
Microcontrolador com Wi-Fi Integrado.

Características:
CPU de 32 bits (geralmente operando a 80MHz ou 160 MHz).
Memória RAM e Flash (para armazenar o código).
Interface Wi-Fi (padrão 802.11).
Interfaces de Entrada/Saída Digital (GPIOs)
Conversor Analógico-Digital (ADC).

Plataforma de Baixo Custo para IoT: Seu preço acessível e a facilidade de integração do Wi-Fi o tornam a escolha ideal para projetos que precisam se conectar à internet, como automação residencial (lâmpadas, sensores, tomadas inteligentes) e monitoramento remoto.

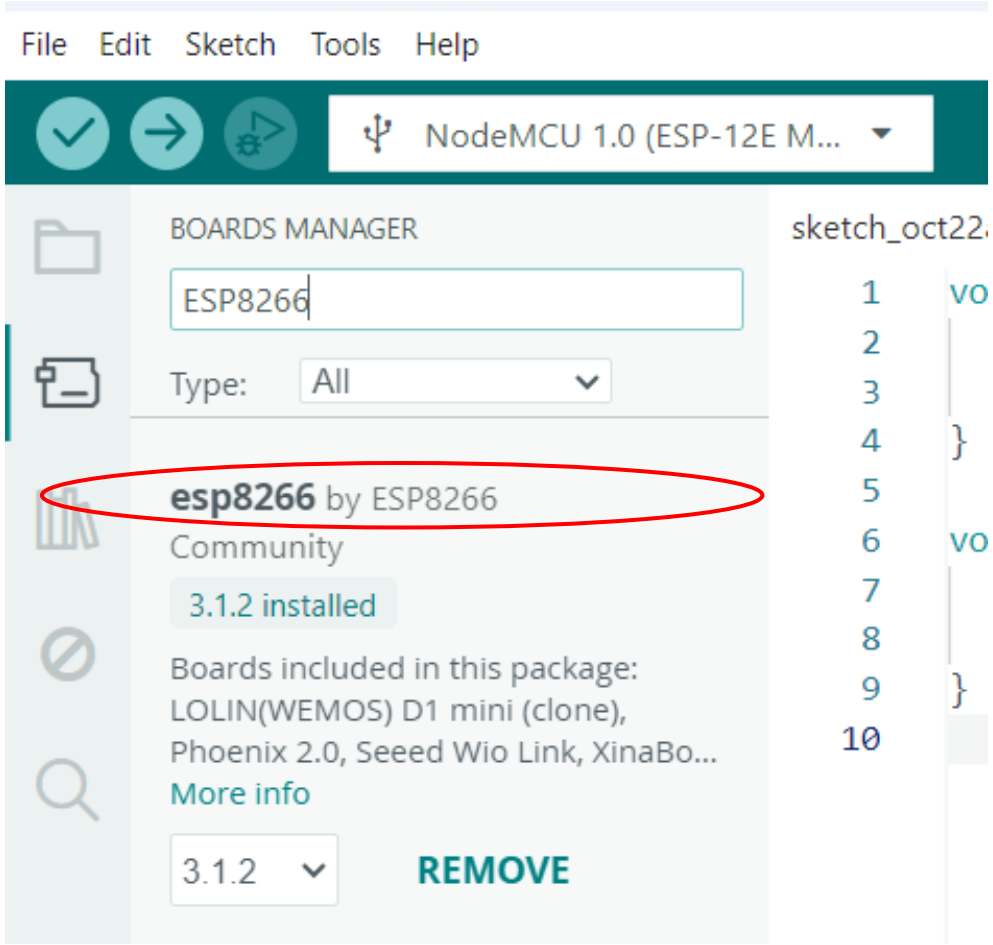
Instalação do ESP8266 no Arduino IDE

Abrir PREFERENCES no menu FILE



Instalação do ESP8266 no Arduino IDE

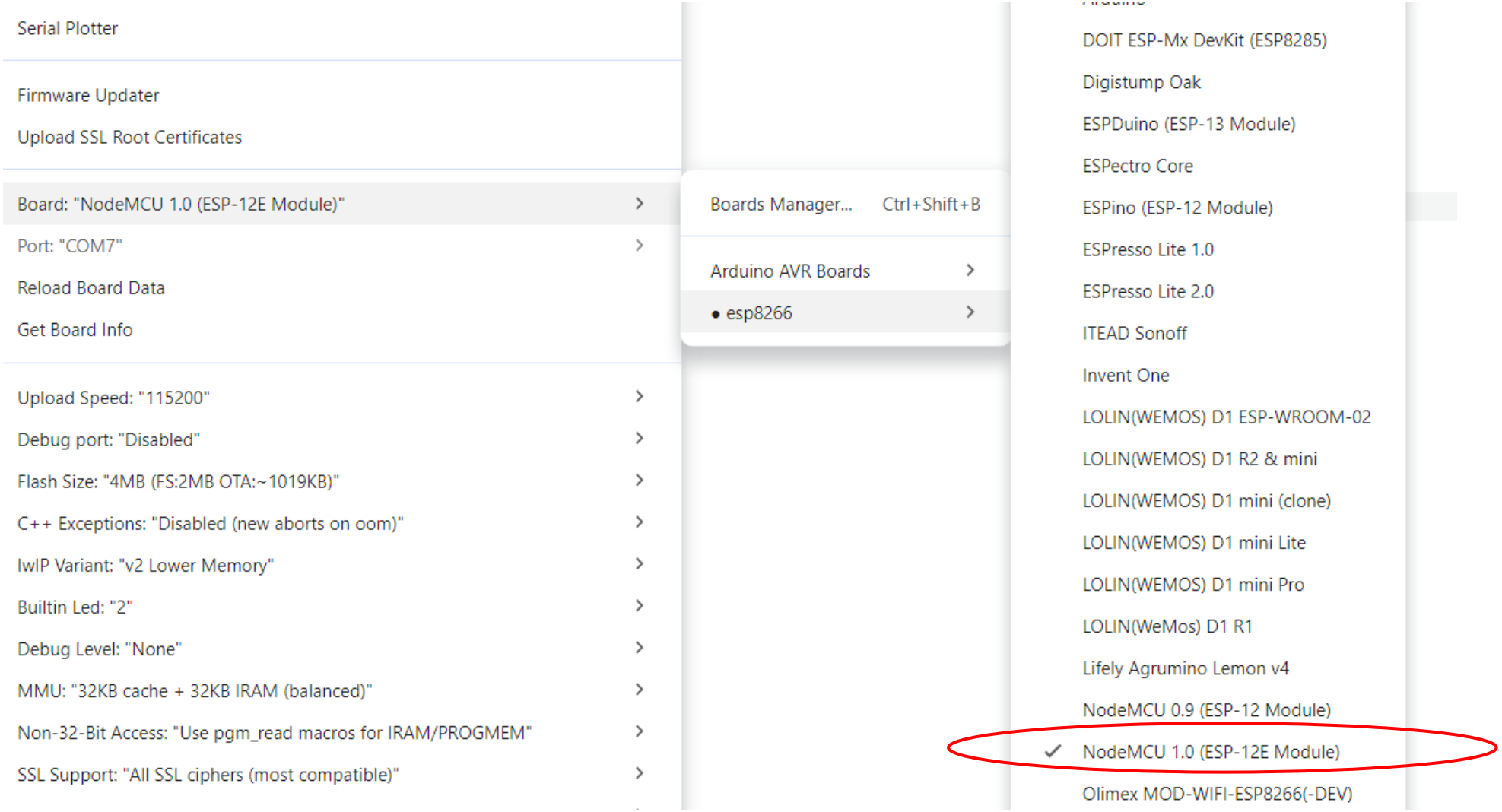
Abrir: TOOLS => BOARD => BOARDS MANAGER



Instalar o ESP8266 by ESP8266

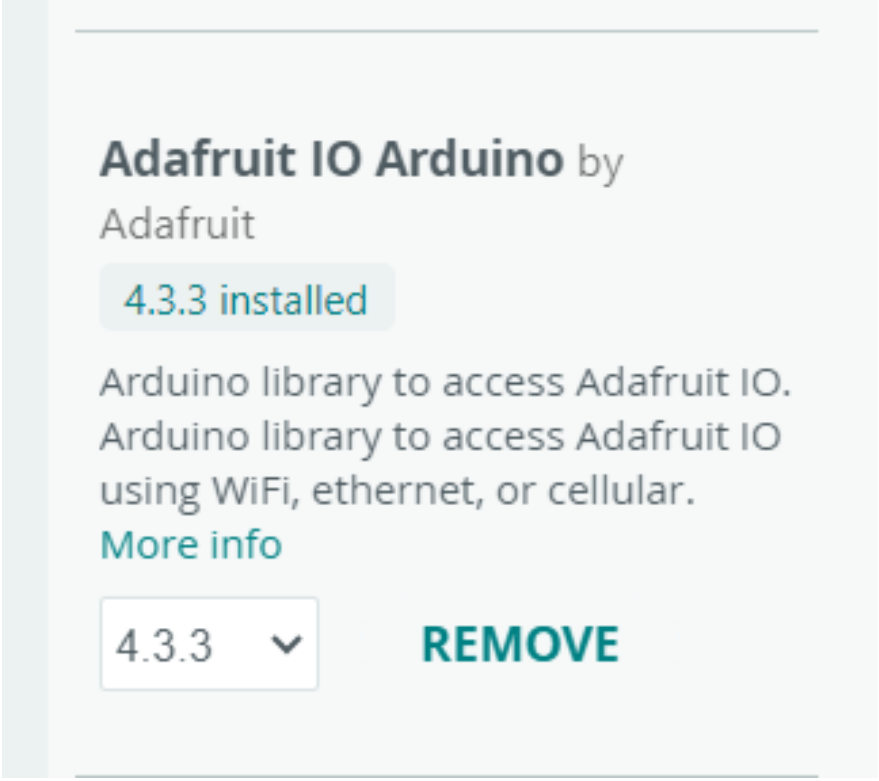
Selecionar a Placa

Abrir: TOOLS => BOARD => esp8266



Instalar Biobliotecas

Abrir: TOOLS => MANAGE LIBRARIES

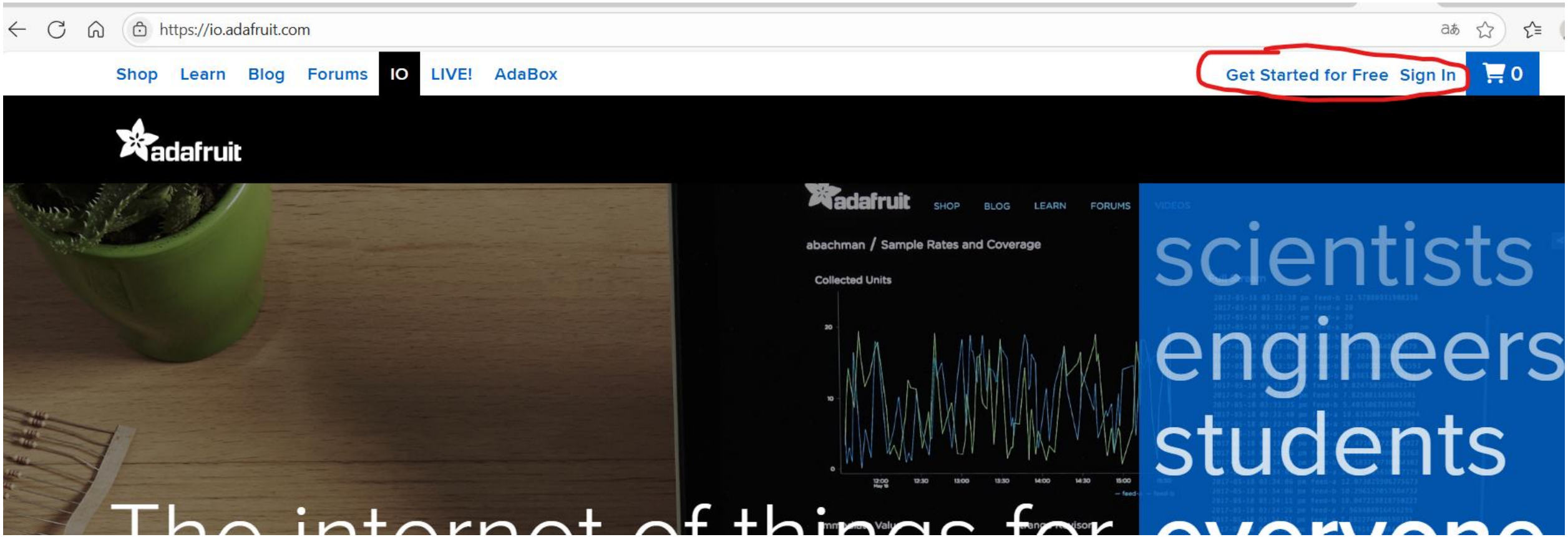


Driver USB

Se necessário, instalar driver para o computador se comunicar com a placa.

https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers

Criar conta na ADAFRUIT (io.adafruit.com)

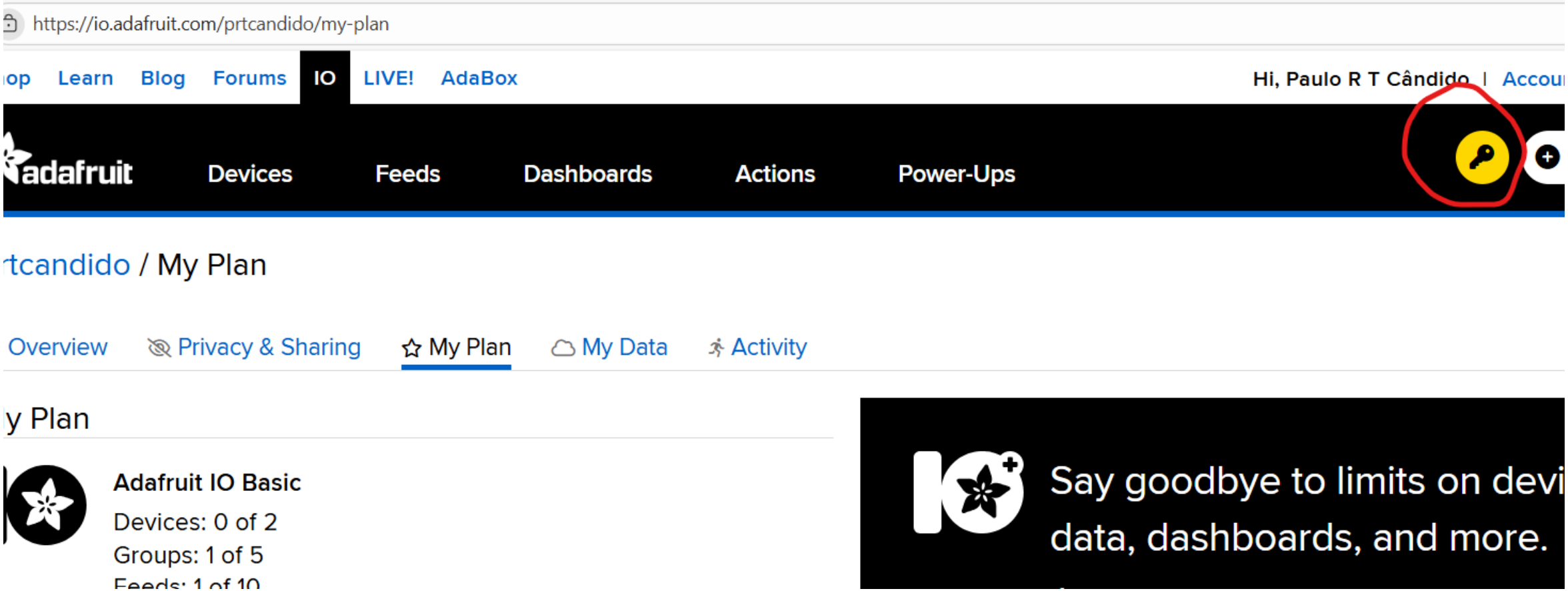


Limitações de conta Free

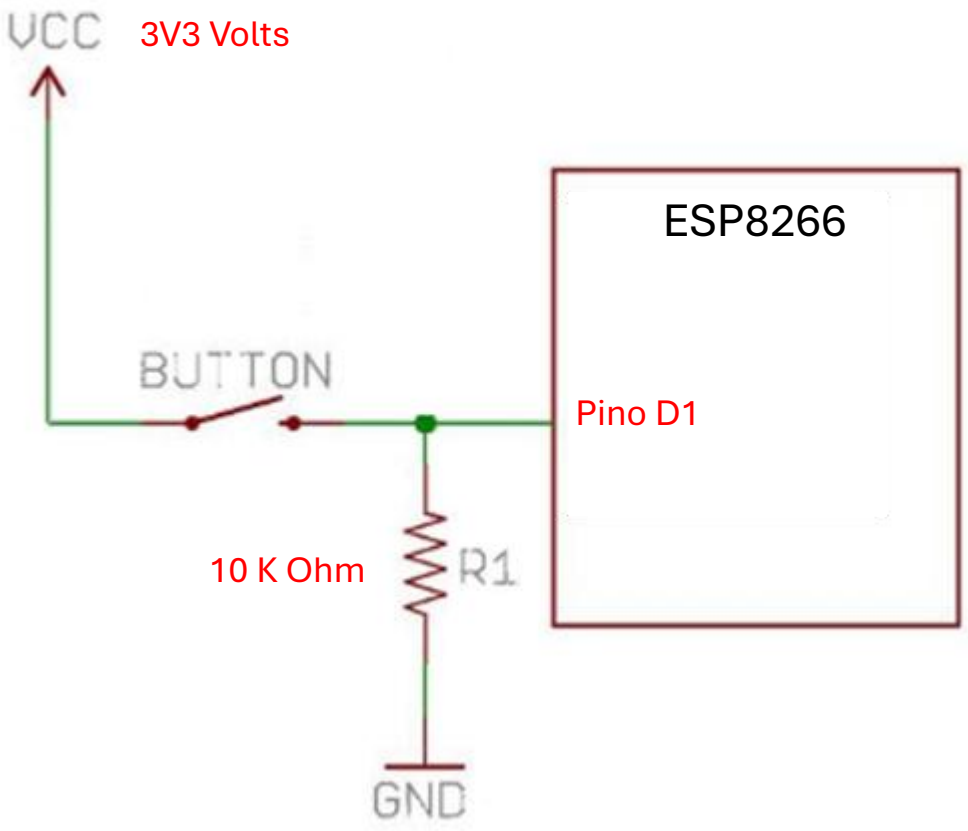
Adafruit IO Basic

Devices: 0 of 2
Groups: 1 of 5
Feeds: 1 of 10
Dashboards: 0 of 5
Data Rate: 30 / min
Data Usage: 0 / min
Storage: 30 days
SMS Rate: 0 / day
SMS Usage: 0 / day

Anotar Usuário e Chave para acesso



Entrada Digital - Botão



Entrada Digital – Botão - Código

```
// Dados na Adafruit
#define IO_USERNAME "xxxxxx" // Usuário
#define IO_KEY "yyyyy" // Chave
// Dados roteador
#define WIFI_SSID “nome wifi" // Informação da SSID do seu roteador
#define WIFI_PASS “senha wifi" // senha de acesso do seu roteador
#include "AdafruitIO_WiFi.h" // inclui biblioteca
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
// Entrada do Botao
#define pinoBotao 5  //D1 na placa corresponde ao GPIO 5 do ESP8266
// Feeds no Adafruit onde o estado do botão será armazenado (publicado)
AdafruitIO_Feed *estadoBotao = io.feed("estadobot");
// Variáveis auxiliares
bool estadoAtualBotao = false;
bool estadoAnteriorBotao = false;
```

Continua

Entrada Digital – Botão - Código

```
void setup() {
  Serial.begin(9600); // Inicia a serial
  pinMode(pinoBotao, INPUT); // Botão é entrada digital

  // Conecta a io.adafruit.com
  Serial.print("conectando com Adafruit IO");
  io.connect();

  // Aguarda conexão
  while(io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println(io.statusText());
}
```

Continua

Entrada Digital – Botão - Código

```
void loop() {

  io.run(); // ver descrição no próximo slide

  // Lê o estado do botao
  if(digitalRead(pinoBotao) == HIGH){
    estadoAtualBotao = true;
  } else {
    estadoAtualBotao = false;
  }
  // retorna se o valor não mudou
  if(estadoAtualBotao == estadoAnteriorBotao)
    return;
  Serial.println(estadoAtualBotao ? "Pressionado" : "Solto");

  // Envia novo estado para a Adafruit
  estadoBotao->save(estadoAtualBotao);

  // armazena o último estado lido
  estadoAnteriorBotao = estadoAtualBotao;
}
```

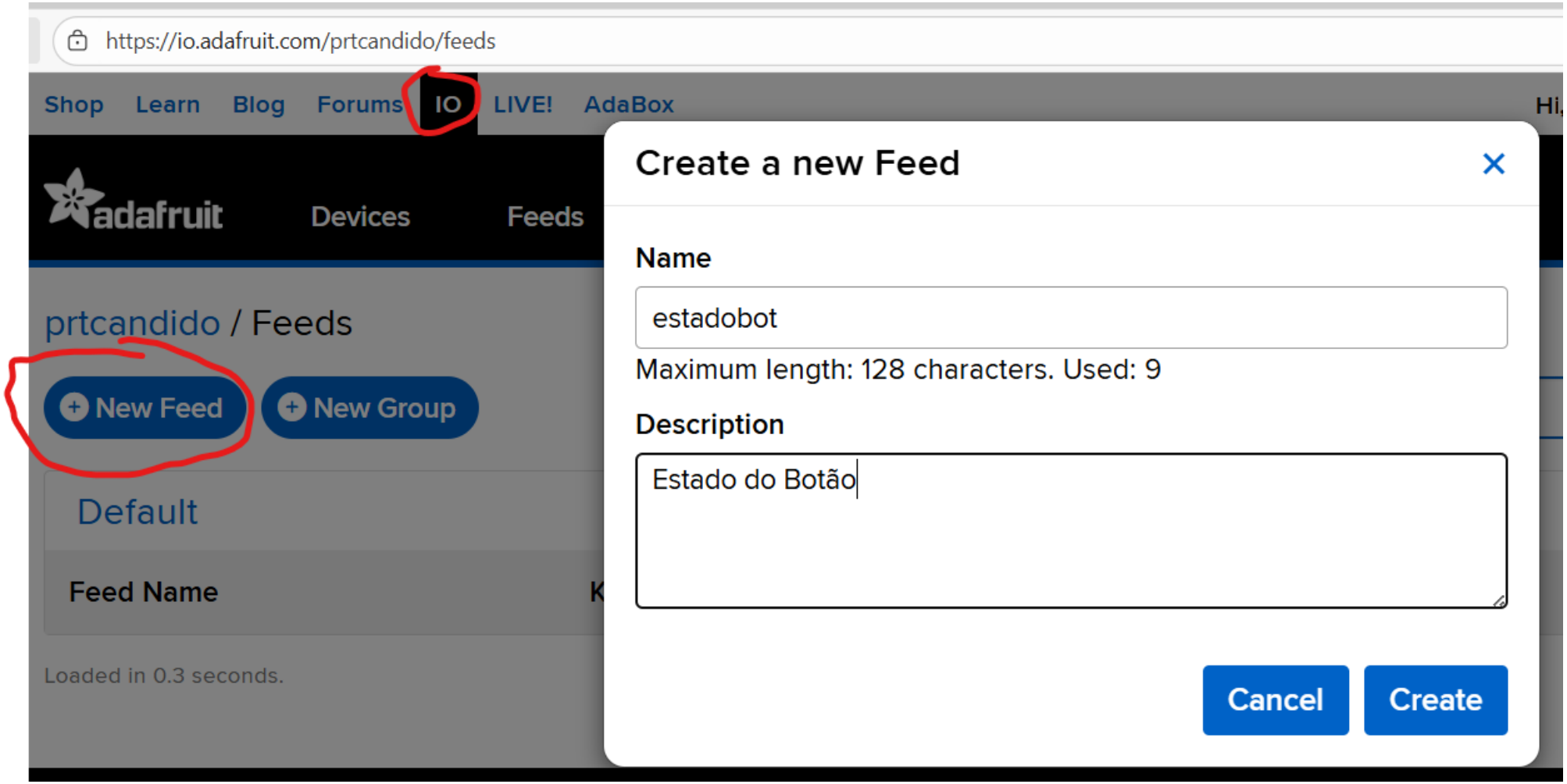
io.run()

O que o `io.run()` faz:

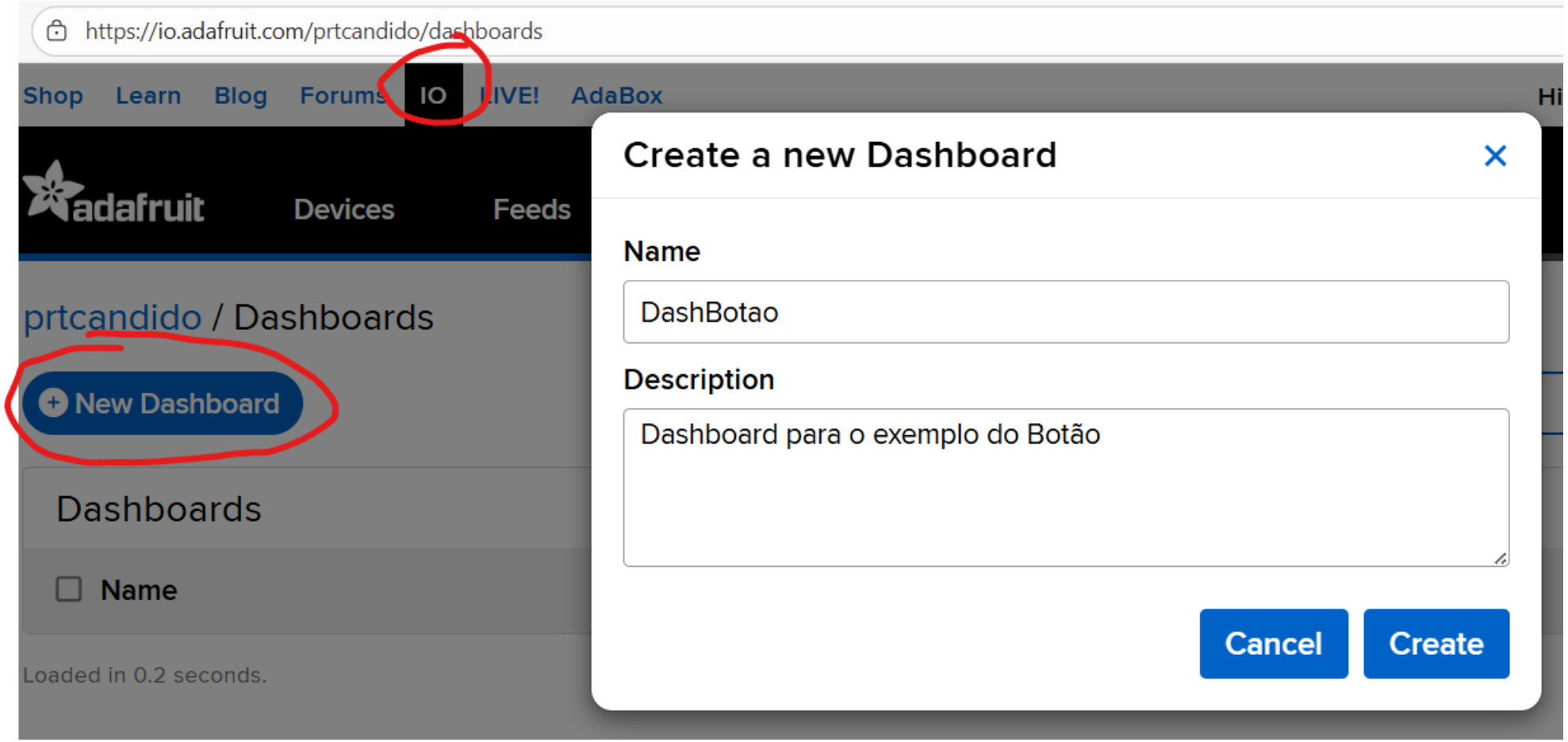
1. **Mantém a Conexão MQTT Ativa:** O protocolo MQTT (que a Adafruit IO usa) exige que o cliente envie e receba pacotes de controle periodicamente para garantir que a conexão não caia. O `io.run()` cuida disso (mantendo o *keep-alive*).
2. **Processa Mensagens Recebidas:** O principal papel do MQTT é receber dados. Quando um dado é publicado em um **Feed** ao qual o seu dispositivo está inscrito, o `io.run()` recebe esse pacote e executa a função de *callback* associada.
   - **Exemplo:** Se você se inscreveu no *feed* `casa/luz`, e alguém publica "ON", o `io.run()` recebe isso e chama a função que você programou para ligar o LED.
3. **Processa Mensagens Pendentes/Saída:** Garante que quaisquer mensagens que o seu dispositivo precisa publicar (enviar) para o Adafruit IO sejam processadas na fila de saída.

Criar o Feed

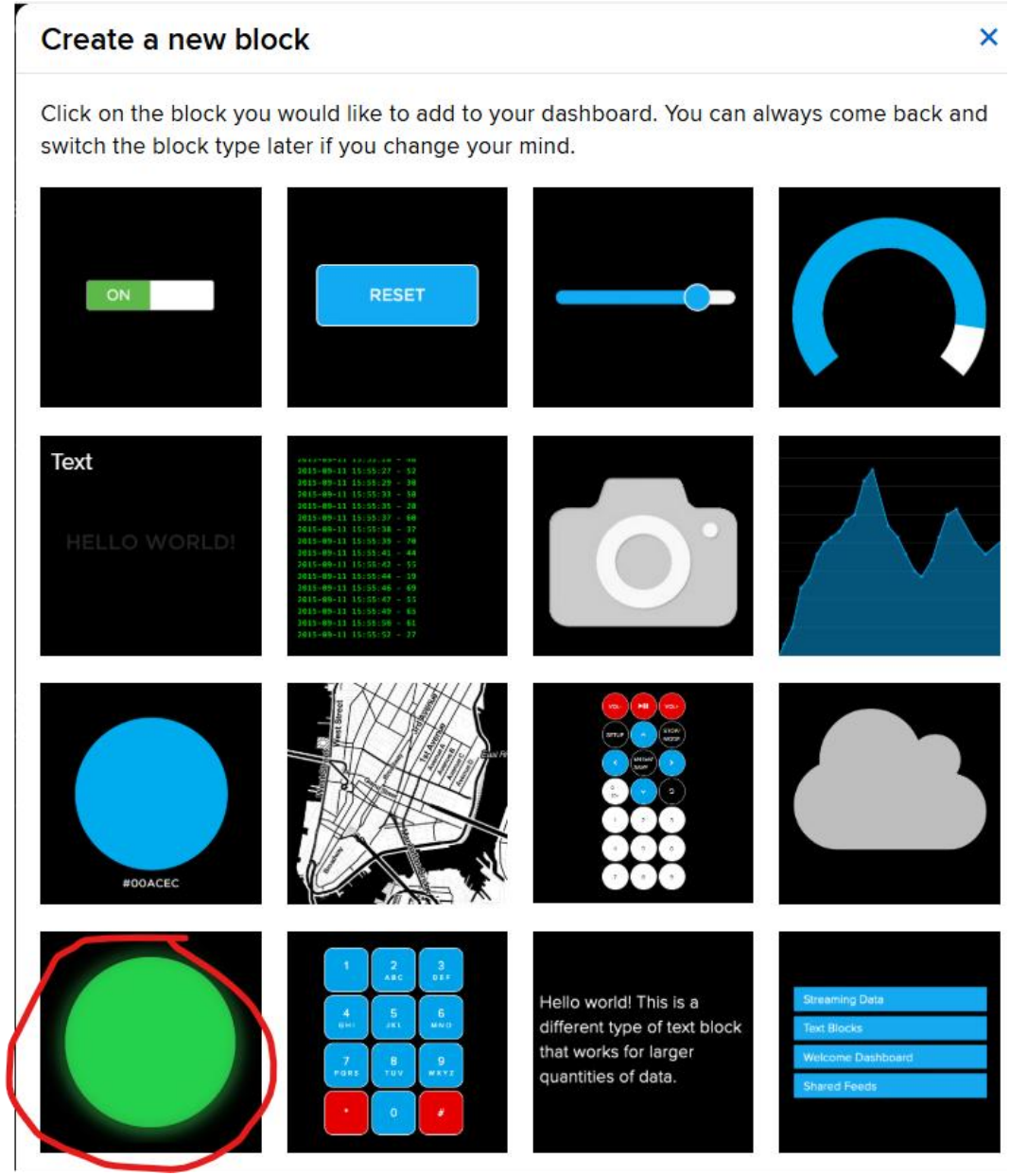
Feed é onde serão armazenados (publicados) os dados gerados pelo botão (mudanças de estado).



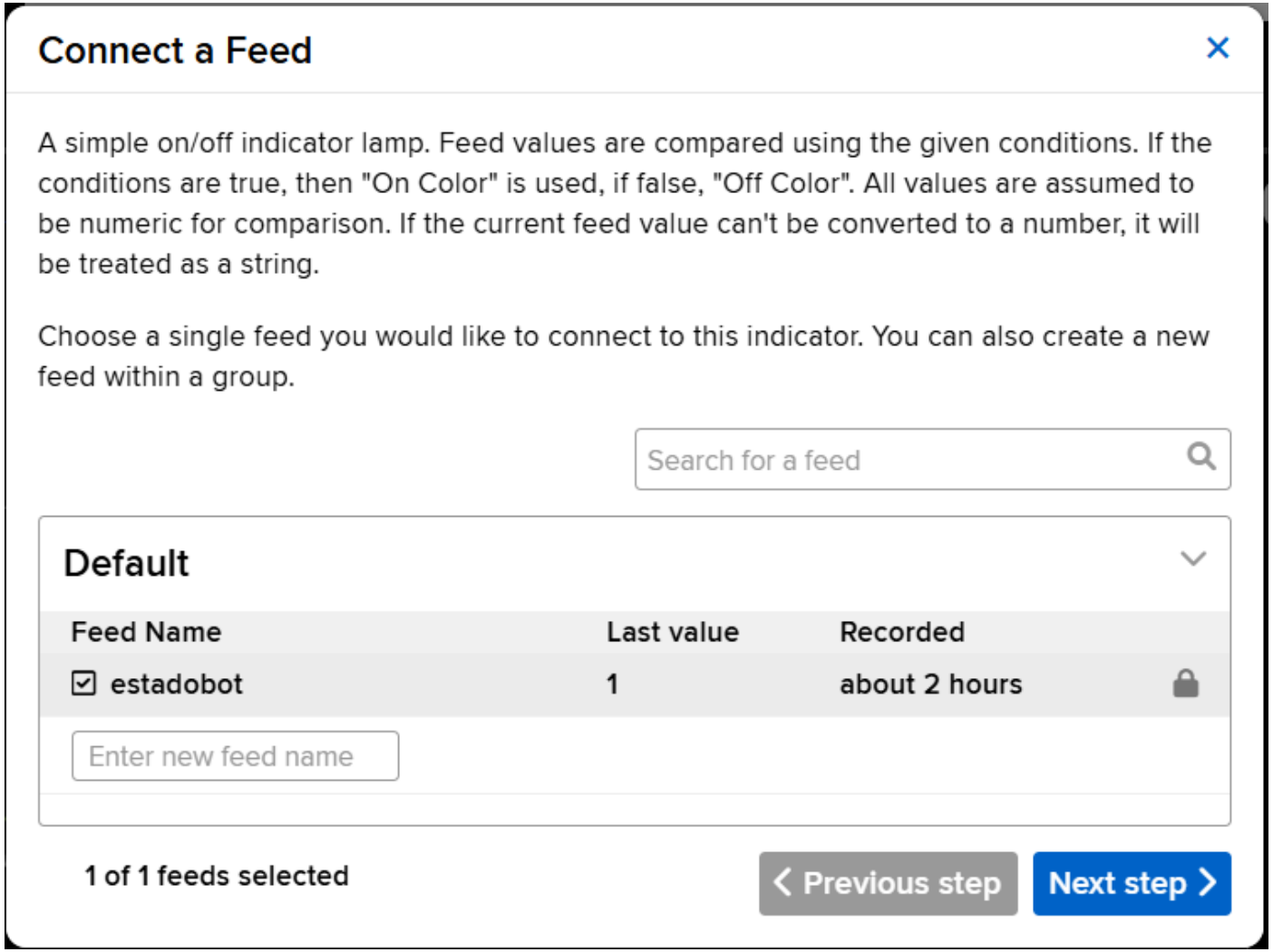
Criar Dashboard



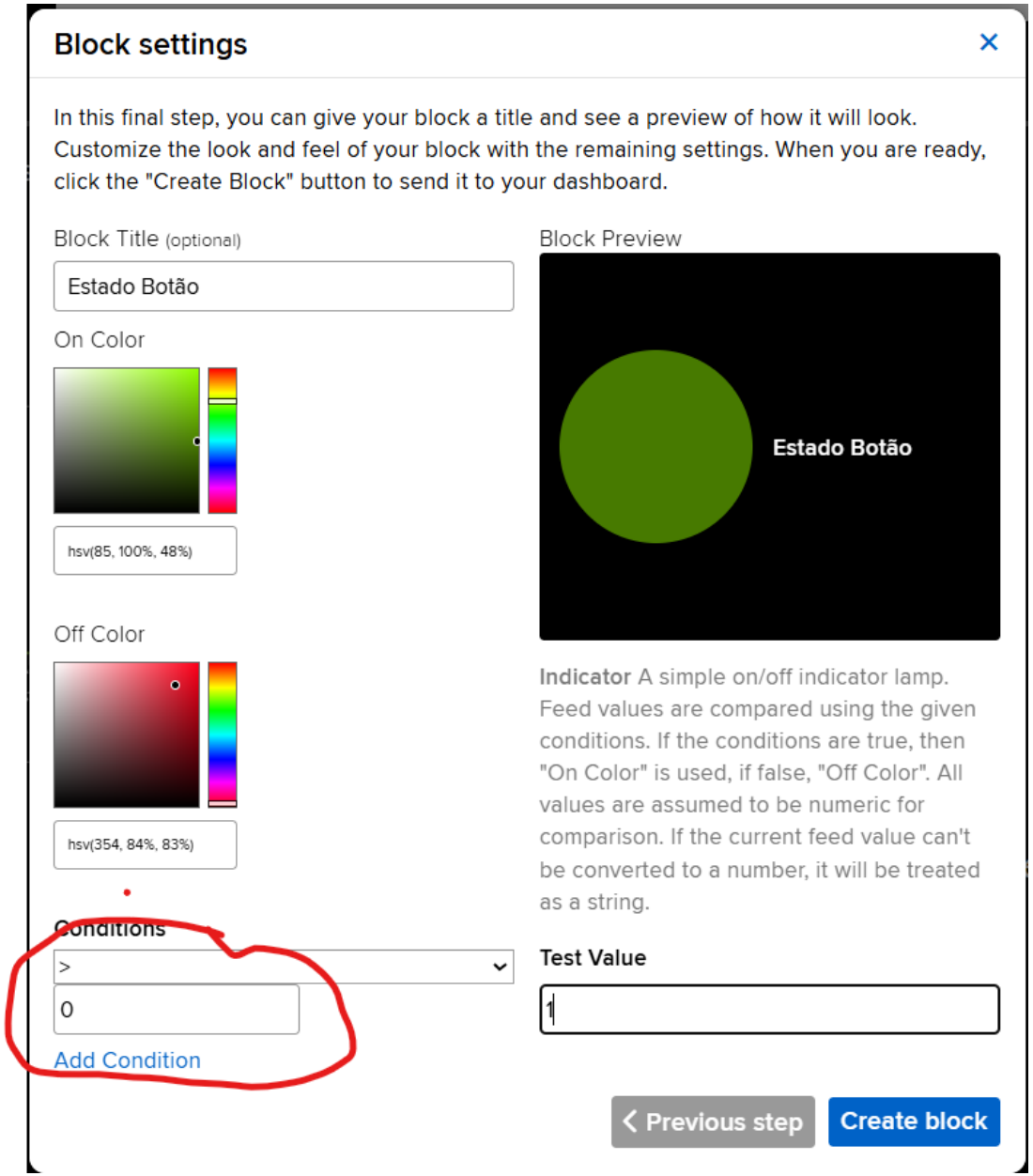
Pressionar New Block e inserir Indicator



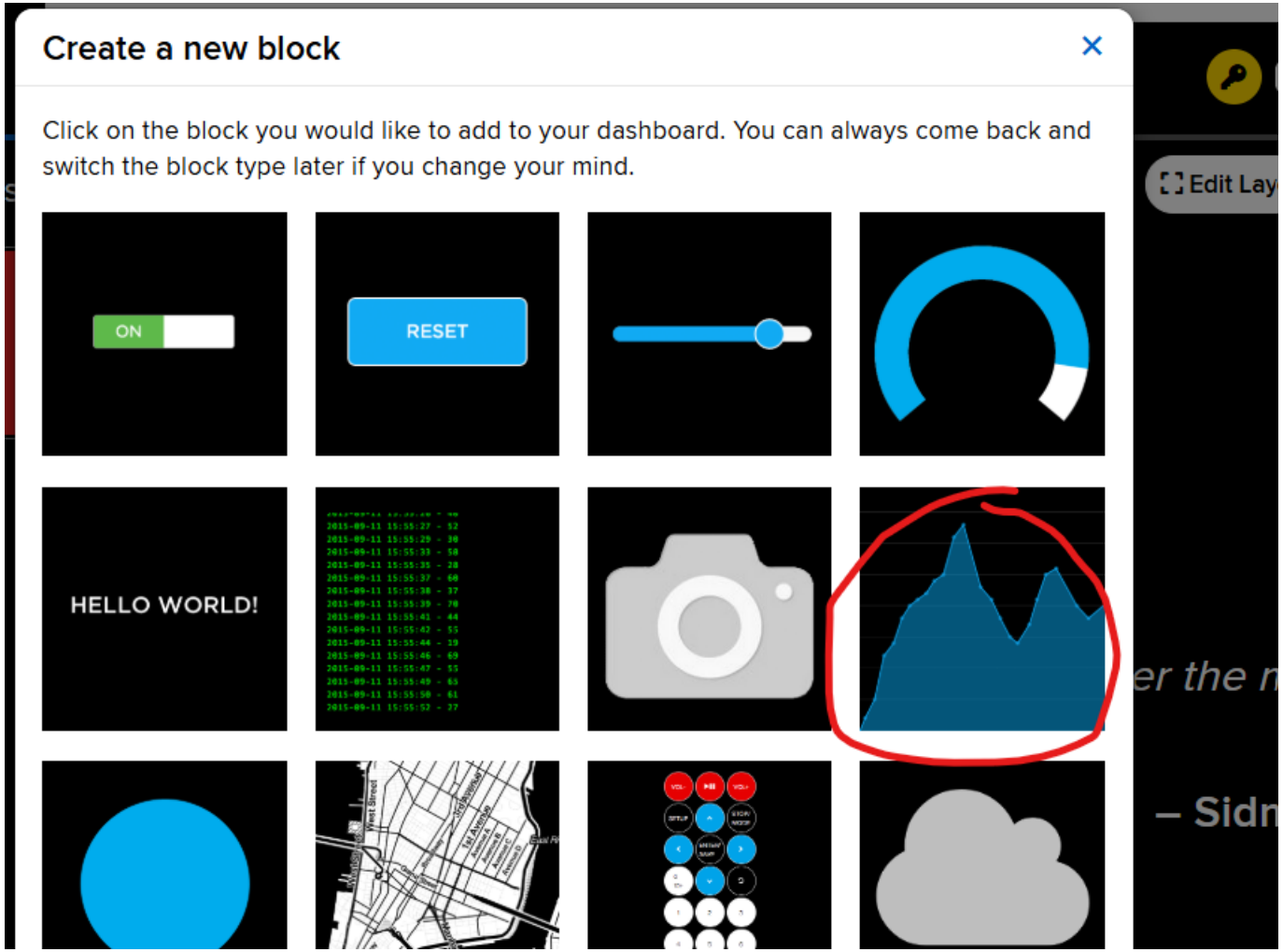
Selecionar o Feed do botão



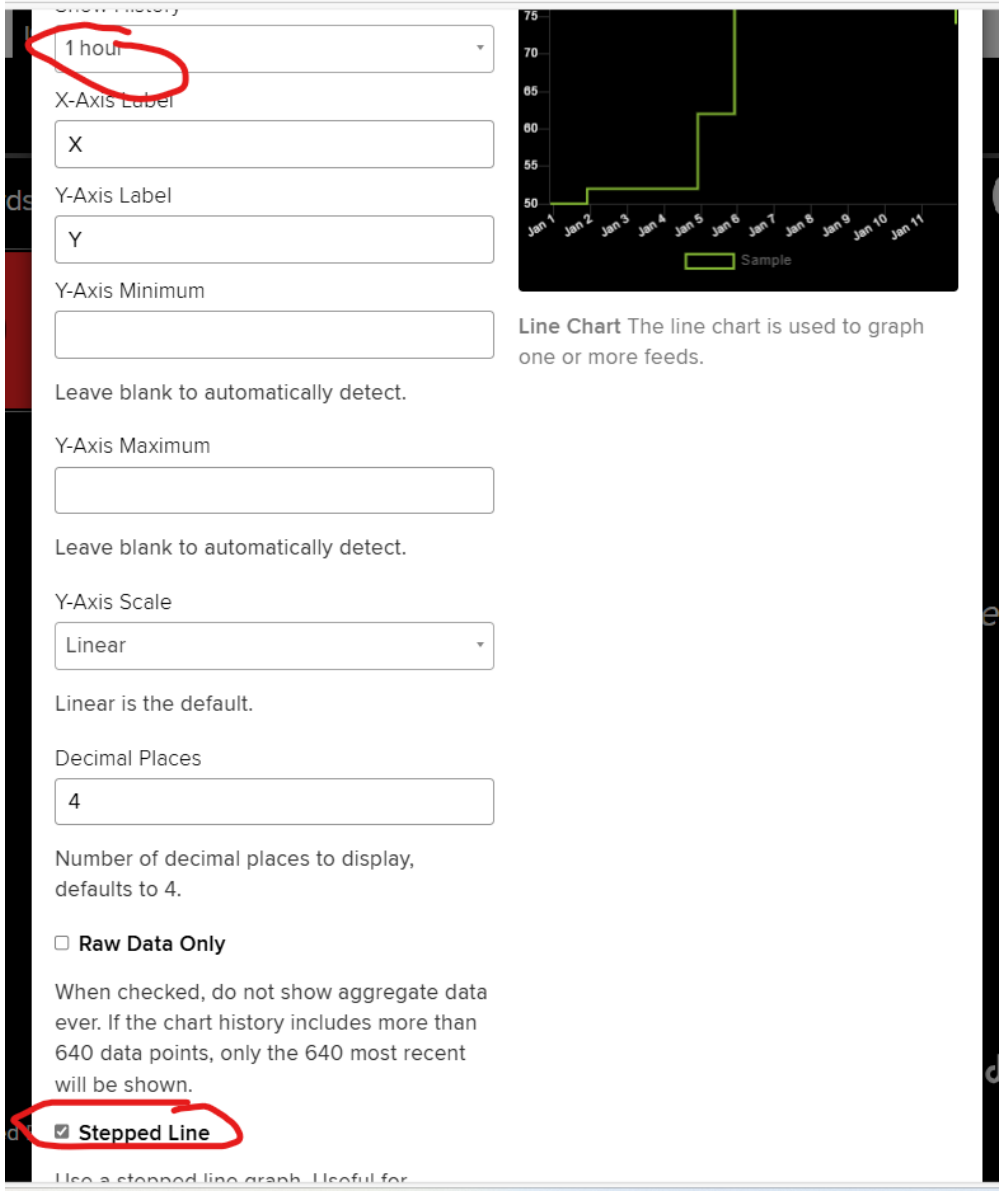
Configurar o bloco



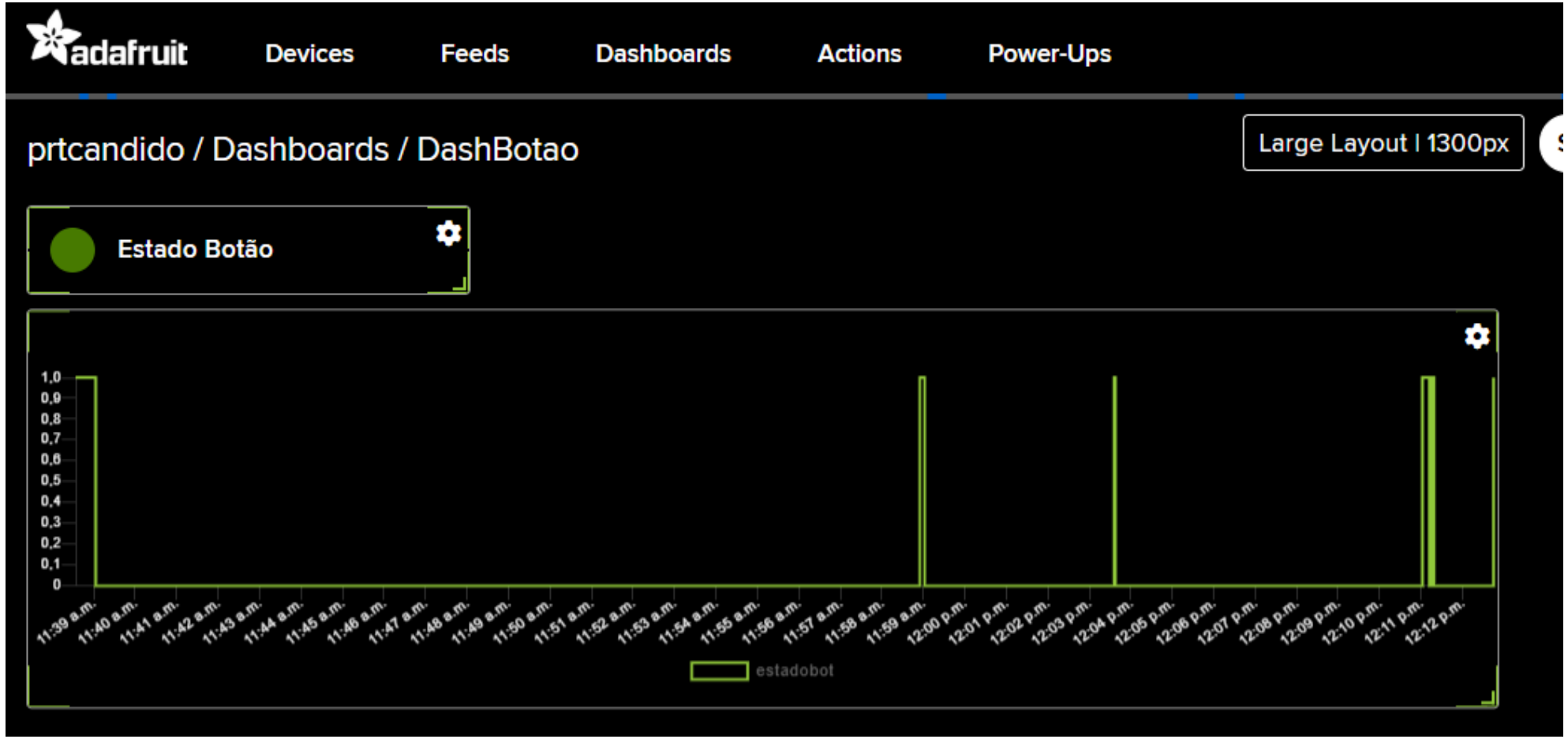
Incluir bloco gráfico (line chart)



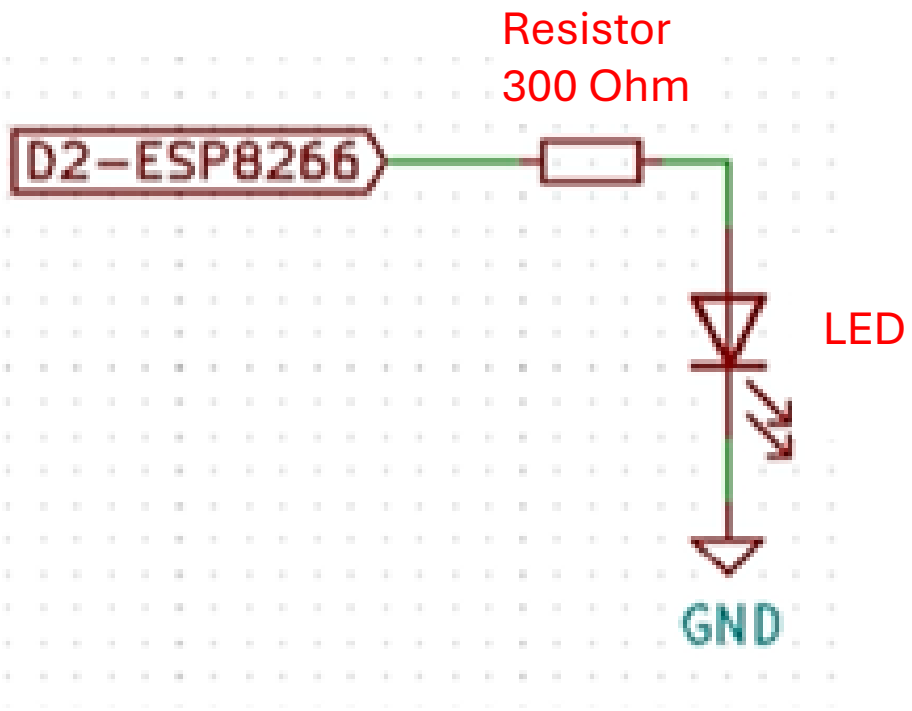
Selecionar o Feed e Configurar o Gráfico



Testar o Dashboard



Saída Digital - Led



Criar em Adafruit o Feed “estadoled”, usado no código a seguir.

Criar Feed e Dashboard

prtcandido / Feeds

Help

New Feed | New Group

Default

Feed Name	Key	Last value	Recorded
estadobot	estadobot	1	26 minutes ago
estadoled	estadoled		less than a minute ago

Criar em Adafruit o Feed “estadoled”, usado no código a seguir.

prtcandido / Dashboards

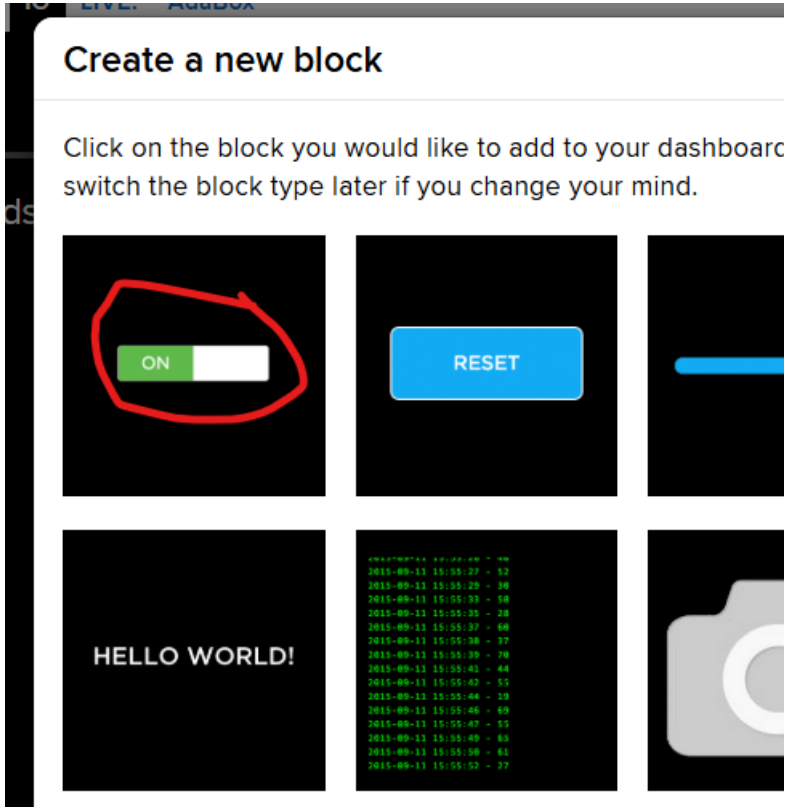
Help

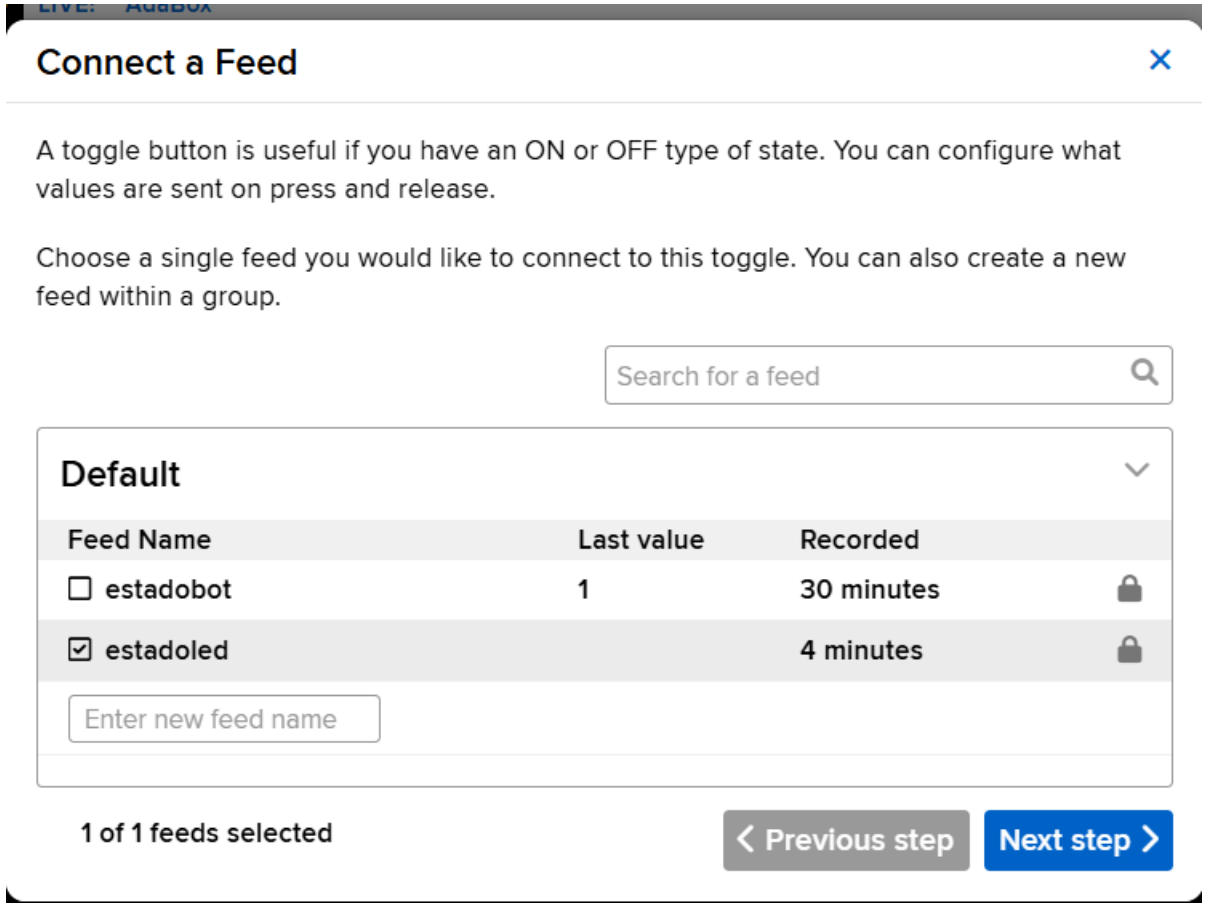
New Dashboard

Dashboards

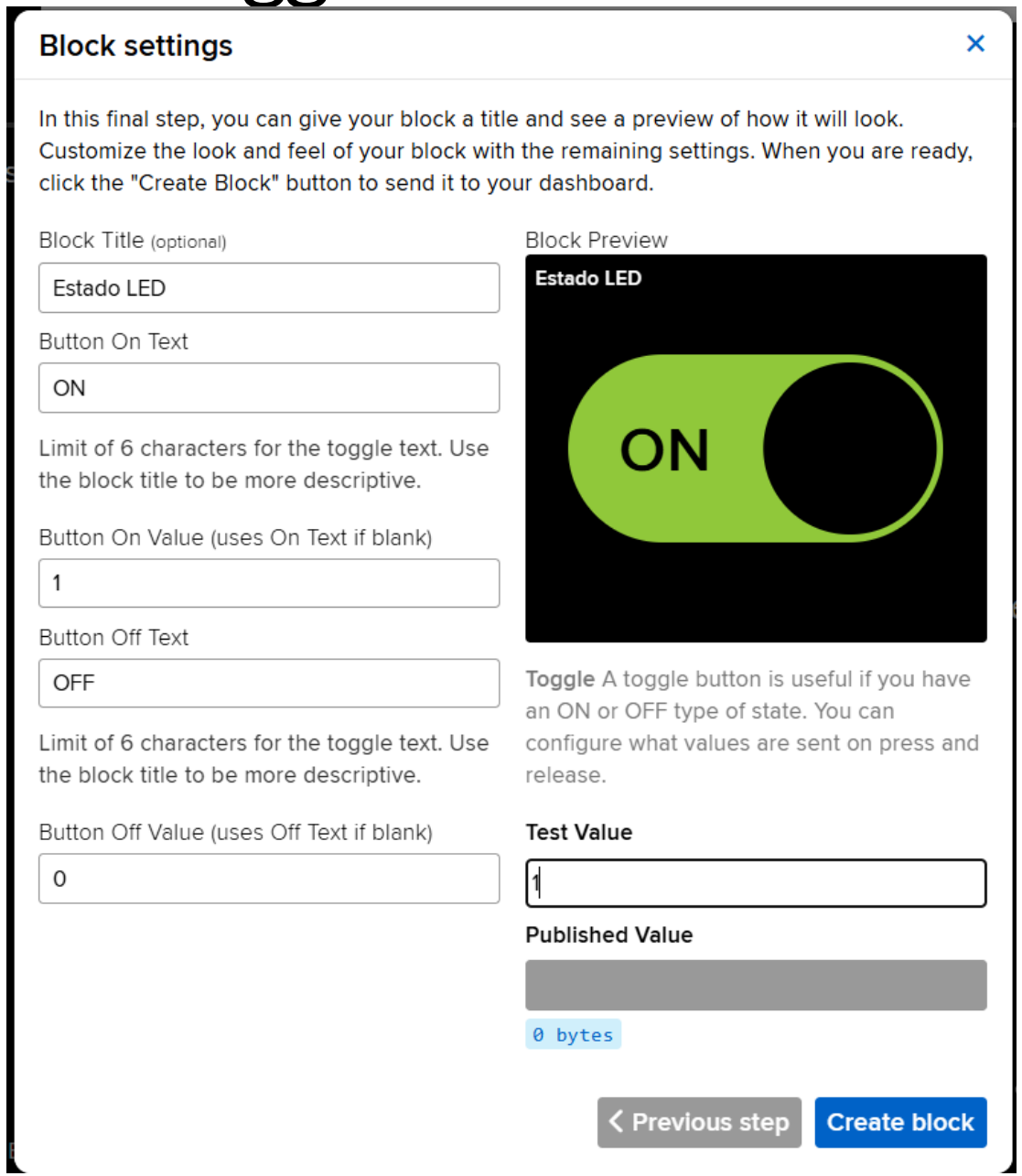
Name	Key	Created At
DashBotao	dashbotao	October 22, 2025
DashLed	dashled	October 22, 2025

Adicionar bloco Toggle ao Dashboard





Adicionar bloco Toggle ao Dashboard



LED - Código

```
// Dados na Adafruit
#define IO_USERNAME “xxxxx" // Usuário
#define IO_KEY “yyyyyy" // Chave
// Dados roteador
#define WIFI_SSID “nome wifi"
#define WIFI_PASS “senha wifi“

#include "AdafruitIO_WiFi.h“

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY,
WIFI_SSID, WIFI_PASS);

// Saída digital do LED
//D2 na placa corresponde ao GPIO 4 do ESP8266
#define pinoLed 4

// Feed do LED – criar no Adafruit
AdafruitIO_Feed *estadoLed =
io.feed("estadoled");
```

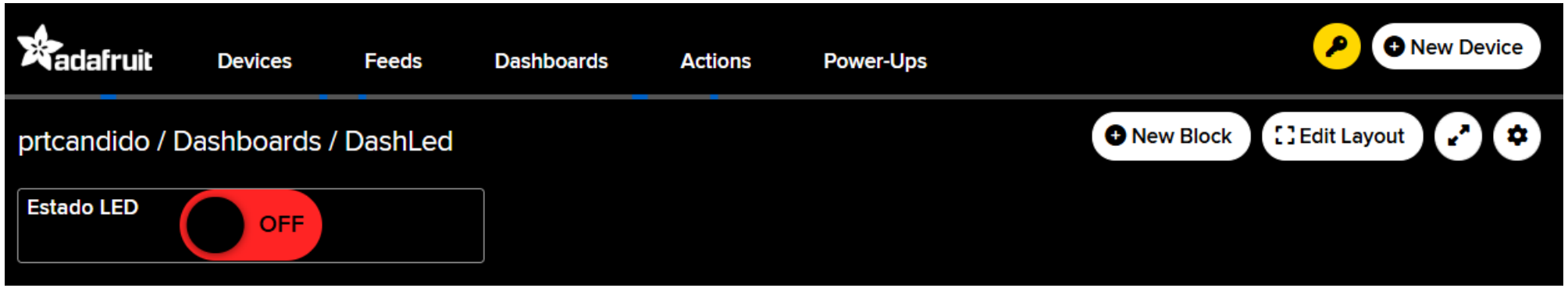
Continua

```
void setup() {
// --- Configuração IO ---
  pinMode(pinoLed, OUTPUT);
  Serial.begin(9600);

  // Sempre que chegar mensagem do Feed (em Adafruit)
  // referente ao Led, a função handleMessage será
  // executada para trata-la.
  estadoLed->onMessage(handleMessage);

  // Conecta a io.adafruit.com
  Serial.print("conectando com Adafruit IO");
  io.connect();

  // Aguarda conexão
  while(io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  // Mensagem na serial quando conectado
  Serial.println();
  Serial.println(io.statusText());

  // Obtem mensagem
  estadoLed->get(); // lê feed estadosaida
}
```

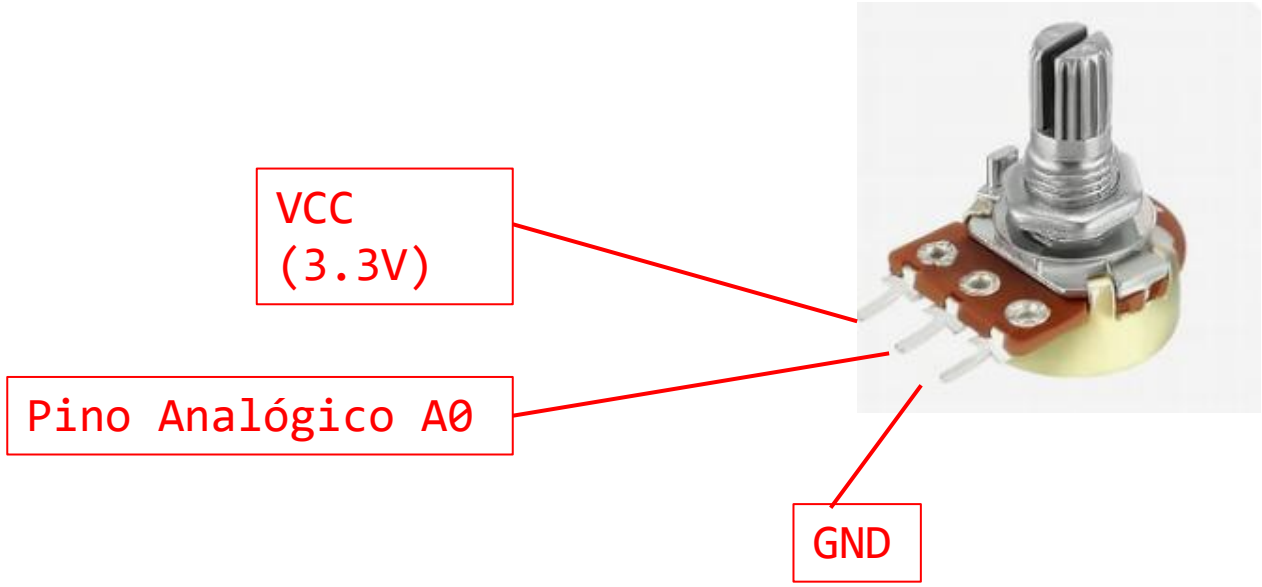
LED - Código

```
void loop() {
  io.run();
}

void handleMessage(AdafruitIO_Data *data) {

  Serial.print("Chegou mensagem da io.adafruit.com");

  // escreve na saída digital o valor recebido
  digitalWrite(pinoLed, data->toPinLevel());
}
```

LED – Alterando Toogle – LED acende ou apaga



Potenciômetro – Leitura Analógica



Criar Feed e Dashboard

prtcandido / Feeds

Help

New Feed | New Group

Default

Feed Name	Key	Last value	Recorded
entradapot	entradapot	5	less than a minute ago
estadobot	estadobot	1	about 1 hour ago
estadoled	estadoled	0	about 1 hour ago

Loaded in 0.26 seconds.

prtcandido / Dashboards

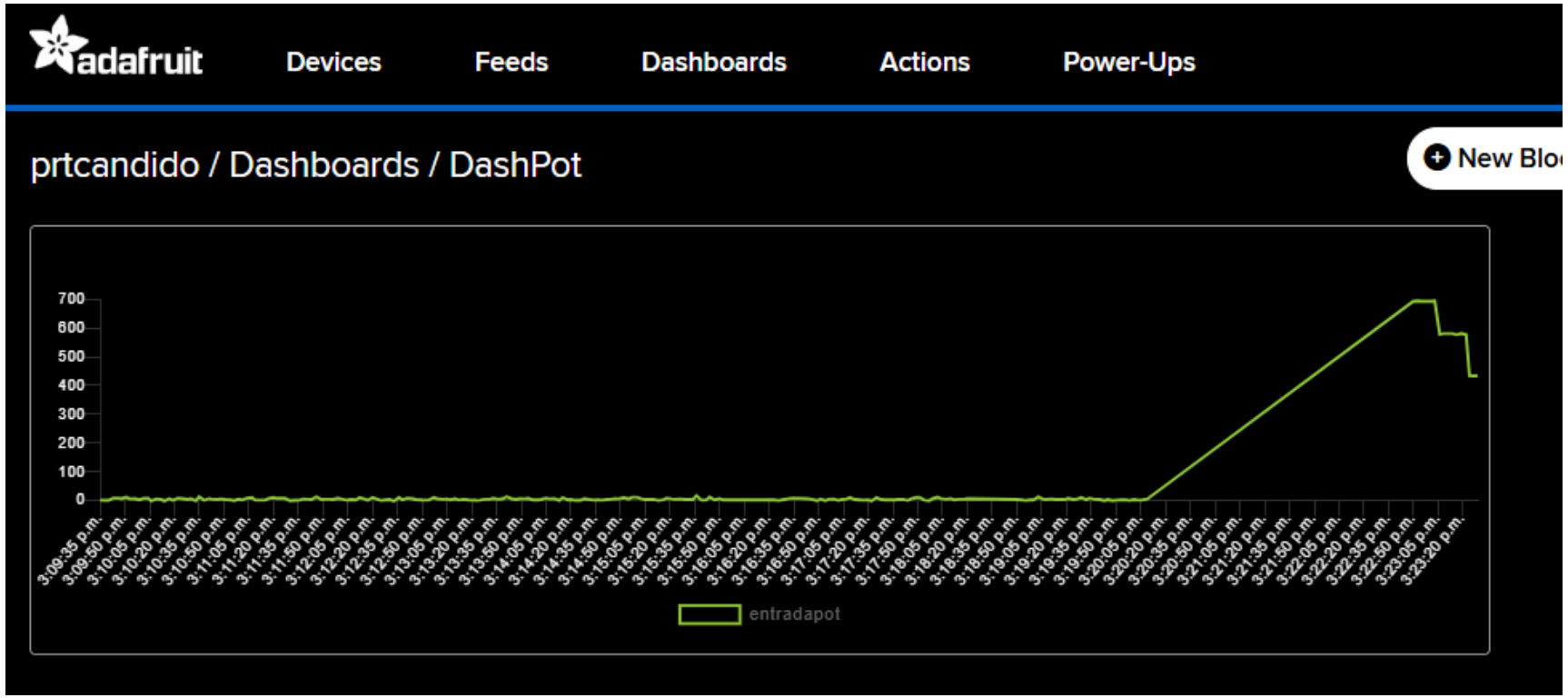
Help

New Dashboard

Dashboards

Name	Key	Created At
DashBotao	dashbotao	October 22, 2025
DashLed	dashled	October 22, 2025
DashPot	dashpot	October 22, 2025

Criar gráfico de linha ao Dashboard associado ao Feed entradapot



Potenciômetro - Código

```
// Dados na Adafruit
#define IO_USERNAME "prtcandido" // Usuário
#define IO_KEY "aio_uKPF29LSx2wfItgVa8CTmMvEakmB" // Chave
// Dados roteador
#define WIFI_SSID "Galaxy A04e 2db4" // Informação da SSID do seu roteador
#define WIFI_PASS "gvbk0357" // senha de acesso do seu roteador
#include "AdafruitIO_WiFi.h" // inclui biblioteca
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
// Pino potenciômetro
#define pinoPot A0  //A0 - Entrada Analógica
// Configurar objeto Feed para publicações dos valores
AdafruitIO_Feed *entradaPot = io.feed("entradapot");


int leituraPot = 0;
```

Continua

Potenciômetro - Código

```
void setup() {
  Serial.begin(9600);
  // Conecta a io.adafruit.com
  Serial.print("conectando com Adafruit IO");
  io.connect();
  // Aguarda conexão
  while(io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  // Mensagem conectado
  Serial.println();
  Serial.println(io.statusText());
}
void loop() {
  io.run();
  // Obtem a leitura do potenciômetro
  leituraPot = analogRead(pinoPot);
  Serial.println(leituraPot);
  // Publica leitura analógica do potenciômetro
  entradaPot->save(leituraPot);

  delay(2500);
}
```